Abstract

I clustered the MNIST handwritten digit dataset using K-Means and Gaussian Mixture Models (GMM) on PCA-reduced features. I tried out different hyperparameter settings, evaluate the results with Adjusted Rand Index (ARI), Normalised Mutual Information (NMI) and Silhouette score, and also look at example digits from each cluster. In the end, a GMM with 20 principal components and 10 full-covariance components turns out to work noticeably better than any of the K-Means setups I tried.

1 Introduction

The MNIST dataset of handwritten digits contains 70 000 grayscale images of digits 0–9, each of size 28×28 pixels. Instead of doing supervised classification, here I look at the unsupervised side: I ignore the labels while training and see how far I can get by just clustering the images.

I compare two standard algorithms from `scikit-learn`: K-Means and Gaussian Mixture Models (GMM). Clustering directly in the 784-dimensional pixel space would be heavy and probably quite noisy, so I first reduce the dimensionality with Principal Component Analysis (PCA). I then judge the quality of the clusters in two ways: with external metrics (ARI and NMI) that compare to the true labels, and with Silhouette score, which only looks at distances. Finally, I also plot example digits per true class and per cluster to get a more intuitive feel for what the models are doing.

2 Data and Preprocessing

2.1 Dataset and train–test split

I load MNIST using `fetch_openml("mnist_784")`. Each image is flattened to a 784-dimensional vector and I scale all pixel values to $[0, 1]$ by dividing by 255. The full dataset has 70 000 examples. I split it into 56 000 training and 14 000 test samples using an 80/20 stratified split, so the class proportions stay roughly the same in both sets. The digit labels are only used later for evaluation; the clustering algorithms never see them during training.

2.2 Visualisation by true class

To get a first impression of the data I plot five randomly selected training images from each digit class. The resulting grid is shown in Figure 1, with one row per class (0–9) and five images per row.

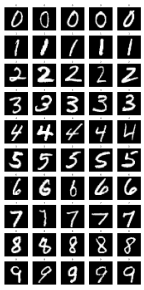

Figure 1: Sample MNIST digits: one row per true class (0–9) and five randomly selected training images per class.

Even this simple plot shows why the problem is not completely easy. Within a single class there is a lot of variation: different stroke thicknesses, different slants, some digits written very neatly and others barely legible. Some "2"s are almost "7"s, and certain "4"s look suspiciously like "9"s. So I do not expect any unsupervised method to exactly recover the label structure, but I hope to see at least a decent amount of alignment.

2.3 Dimensionality reduction with PCA

Before clustering I apply PCA to the normalised pixels. I keep $d \in \{20, 50, 100\}$ principal components and run the clustering in this reduced space. Table 1 shows how much variance each choice captures.

Table 1: Explained variance for different PCA dimensions.

Components	Explained variance
20	0.645
50	0.825
100	0.915

So with 20 components I already keep about two thirds of the variance; with 100 components it goes up to just over 91%. This reduction makes the clustering much cheaper to run and also acts as a form of denoising, because I drop low-variance directions that often correspond to small local pixel changes rather than meaningful structure.

3 Methods

3.1 K-Means clustering

For K-Means I use `sklearn.cluster.KMeans` with Euclidean distance, `max_iter=300`, `n_init=10` and a fixed random seed for reproducibility. I treat two quantities as hyperparameters:

- the number of clusters $k \in \{10, 15, 20\}$,
- the PCA dimensionality $d \in \{20, 50, 100\}$.

For every (d, k) pair I fit K-Means on the PCA-transformed training data and compute ARI, NMI and Silhouette score on that same split. I then pick the configuration with the highest training ARI as the "best" K-Means model, refit it on the training set, and report results on the test set.

3.2 Gaussian Mixture Models

For GMM I use `sklearn.mixture.GaussianMixture`, trained with the expectation–maximisation algorithm. Here I sweep over three hyperparameters:

- the number of mixture components $m \in \{10, 15, 20\}$,
- the PCA dimensionality $d \in \{20, 50, 100\}$,
- the covariance structure: `full` or `diag`.

I set `reg_covar=1e-6` to avoid numerical issues in the covariance matrices and again fix the random seed. As with K-Means, I select the configuration with the highest training ARI and then evaluate that model on the test set.

3.3 Evaluation metrics

I rely on three standard clustering metrics. The Adjusted Rand Index (ARI) measures how similar the cluster assignments are to the true labels, correcting for chance; random clusterings end up near 0 and a perfect match gives 1. Normalised Mutual Information (NMI) measures how much information the cluster labels share with the true labels, again between 0 and 1 and invariant to permutations of cluster IDs. The Silhouette score is an internal metric that does not use labels at all. For each point it compares the average distance to its own cluster with the distance to the closest other cluster. Values near 1 mean the point is much closer to its own cluster than to others, values around 0 mean clusters overlap, and negative values indicate obvious misassignments.

4 Results

4.1 K-Means performance

Table 2 summarises the K-Means grid search on the training data (all scores are computed in the PCA space).

Table 2: K-Means training performance for different PCA dimensions and numbers of clusters.

PCA	k	ARI	NMI	Silhouette
20	10	0.357	0.490	0.124
20	15	0.374	0.532	0.129
20	20	0.366	0.562	0.135
50	10	0.363	0.497	0.085
50	15	0.378	0.536	0.091
50	20	0.350	0.547	0.096
100	10	0.364	0.498	0.071
100	**15**	**0.379**	**0.537**	**0.075**
100	20	0.350	0.548	0.080

Figure 2 shows training ARI as a function of k for each PCA dimension. Performance peaks at $k = 15$, while $k = 20$ tends to fragment digits into smaller style-based subclusters without improving agreement with the labels.

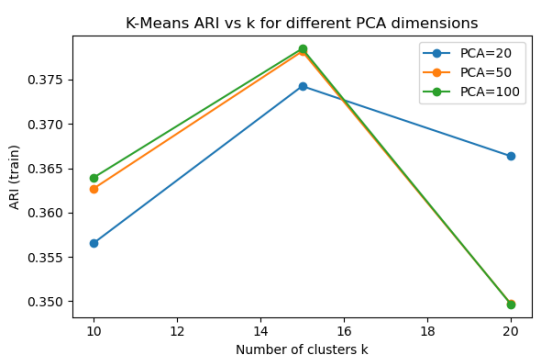

Figure 2: K-Means training ARI versus number of clusters k for different PCA dimensions.

The best configuration by training ARI uses $d = 100$ principal components and $k = 15$ clusters. After refitting this model and evaluating on the test set, I obtain the metrics in Table 3.

Train and test scores are almost identical, so at least in this setup K-Means does not really overfit. ARI and NMI are in a reasonable range: the clusters clearly relate to the labels, but there is plenty of disagreement. Silhouette scores are small but positive, consistent with overlapping clusters. ARI/NMI peak at $k = 15$; increasing to $k = 20$ tends to over-fragment the data.

Table 3: Best K-Means configuration (PCA=100, $k = 15$).

Split	ARI	NMI	Silhouette
Train	0.3785	0.5366	0.0752
Test	0.3802	0.5380	0.0777

4.2 GMM performance

Table 4 shows selected GMM configurations on the training data. The best model uses $d = 20$ PCA components, 10 mixture components and full covariance matrices.

Table 4: Selected GMM training results.

PCA	Comp.	Cov.	ARI	NMI	Silhouette
20	10	full	**0.488**	**0.637**	0.090
20	15	full	0.472	0.666	0.081
20	20	full	0.413	0.637	0.086
20	10	diag	0.339	0.521	0.092
50	10	full	0.429	0.582	0.044
50	20	full	0.366	0.607	0.044
100	10	full	0.369	0.531	0.027
100	20	full	0.326	0.566	0.030

Figure 3 shows training ARI for full-covariance GMMs as the number of mixture components varies. The best performance occurs at 10 components with $d = 20$ PCA, and increasing either the number of components or the PCA dimension generally reduces ARI, consistent with over-fragmentation and harder covariance estimation in higher dimensions.

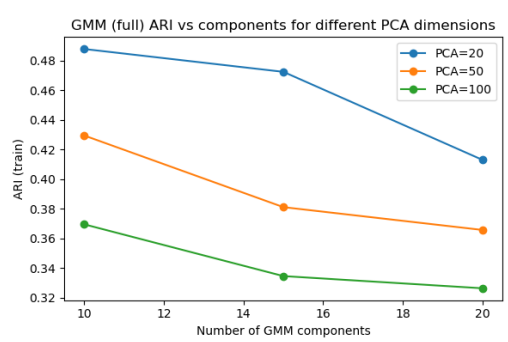

Figure 3: Full-covariance GMM training ARI versus number of mixture components for different PCA dimensions.

Refitting this best configuration and evaluating on the test set gives the results in Table 5.

Compared with K-Means, GMM is clearly ahead. On the test set the best GMM improves ARI by about 0.11 and NMI by about 0.10. Silhouette also goes up slightly, meaning the clusters are a bit more compact and separated. Full covariance matrices almost always beat diagonal ones, which tells me that correlations between PCA features really matter for capturing the shape of the digit clusters. The fact that the best scores happen at $d = 20$ is also telling: when I move to 50 or 100

Table 5: Best GMM configuration (PCA=20, 10 full-covariance components).

Split	ARI	NMI	Silhouette
Train	0.4878	0.6366	0.0905
Test	0.4921	0.6429	0.0943

components, both ARI and Silhouette drop. My guess is that in those higher dimensions the model has to estimate many more covariance parameters and starts to chase noise rather than signal.

4.3 Direct comparison

Table 6: Comparison of best K-Means and best GMM (test set).

Model	ARI	NMI	Silhouette
K-Means (PCA=100, $k = 15$)	0.3802	0.5380	0.0777
GMM (PCA=20, 10 full)	0.4921	0.6429	0.0943

GMM wins on all three metrics. Because training and test scores are very close for both models, this difference is unlikely to be a pure overfitting effect. It is more about the modelling assumptions: GMM is simply better equipped to describe the structure of the digits in this feature space.

5 Qualitative Analysis

To see what the clusters actually look like, I visualise digits assigned to each GMM component for the best configuration. In Figure 4 each row corresponds to one cluster and shows five randomly chosen training digits from that cluster.

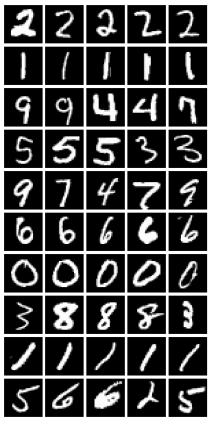

Figure 4: Example digits per GMM cluster for the best configuration (20 PCA components, 10 full-covariance components). Each row corresponds to one cluster and shows five randomly sampled training images.

Some clusters are almost "pure" in terms of digit identity. For example, one row is basically all "0"s, and another row contains clean, vertical "1"s. Other clusters are clearly mixed: it is common to see both "4"s and "9"s in the same row, or a blend of rounded "3"s and "5"s. There are also digits that get spread across several clusters, especially "2" and "7". The model tends to separate those according to style: curved "2"s live in different clusters from more angular "2"s, and so on.

This qualitative picture matches the quantitative metrics quite nicely. When the handwriting is clear and consistent, a cluster lines up well with a single digit label and helps push ARI and NMI up. When the writing is ambiguous or two digits look almost the same, the model has little choice but to mix labels within a cluster, which limits the scores. The moderate Silhouette values reflect the fact that in the PCA space many clusters sit quite close to each other, and points near the boundaries are not strongly attached to one cluster or another.

6 Discussion and Conclusion

Overall, both K-Means and GMM manage to pick up a fair amount of structure in the MNIST digits, but GMM is clearly the better fit here. The main reason is the shape of the clusters. K-Means effectively assumes that clusters are spherical and share the same variance, so it carves the space into Voronoi cells around k centroids. A GMM with full covariance matrices, on the other hand, represents each component as an ellipsoid with its own orientation and scale. In the PCA space the digit manifolds are anything but spherical, so being able to tilt and stretch clusters in different directions makes a big difference.

The experiments also underline how important the choice of hyperparameters and the PCA dimensionality is. For K-Means, moving from $k = 10$ to $k = 15$ helps because the algorithm can split a single digit into multiple clusters that capture different styles. Increasing further to $k = 20$ mostly fragments things without major gains. For GMM, 10 components—roughly matching the number of digit classes—already works best; extra components mainly break digits into subgroups. The interaction with PCA is particularly interesting: K-Means copes fairly well with higher-dimensional PCA spaces, whereas GMM clearly prefers the compact 20-dimensional representation, where estimating the covariance matrices is easier and less prone to overfitting.

In summary, the best model I found is a GMM with 20 PCA components, 10 full-covariance components and a small covariance regulariser. On the test set it reaches ARI ≈ 0.49, NMI ≈ 0.64 and Silhouette ≈ 0.09, comfortably beating the best K-Means model. The main takeaway for me is that combining a simple dimensionality reduction step with a probabilistic clustering model that has flexible cluster shapes is already enough to get a reasonably good unsupervised understanding of handwritten digits.

Self-reflection

Working through this experiment gave me a better feel for how unsupervised methods behave on real data. It was interesting to see that even without label information the models still discovered clusters that roughly lined up with the digit classes, but also where they struggled because of ambiguous handwriting. I also got used to reading ARI, NMI and Silhouette together instead of staring at a single number. ARI and NMI told me how well the clusters matched the labels, while Silhouette said something about how tidy the clusters were in the feature space. Finally, I was reminded how much impact preprocessing and a sensible hyperparameter grid can have. Simple decisions like applying PCA and keeping the number of components moderate mattered more than fiddling with obscure settings, and using tools like `scikit-learn` helped me focus on interpreting the results instead of fighting with the implementation.